Artifact Submission Abstract — ASE 2026 Artifact Evaluation

ReproBreak: A Dataset of Reproducible Web Locator Breaks

Link to the accepted paper

Author version: included in the artifact archive as `2026_ase_locator_break_dataset.pdf`, and available at https://arxiv.org/pdf/2605.12158.

Purpose

ReproBreak is the first dataset of reproducible web locator breaks for the Cypress and Playwright GUI-testing frameworks. A locator break happens when a structural change in the application under test makes a test locator, such as a CSS selector, miss its target element. The test breaks although the application still behaves correctly.

The artifact contains:

- a SQLite database with 9,572 locator changes mined from 359 open-source repositories and 449 validated locator breaks across four projects. The Docker reproduction environments are stored inside the database.
- `reproduce.py`, which re-executes any of the 449 breaks in a Docker environment.
- validation scripts that check the paper's claims mechanically.
- the complete three-phase pipeline used to build the dataset. It can extend the dataset to more repositories and frameworks.

Badge

We apply for **Artifacts Available** and **Artifacts Reusable**.

Available. The complete artifact is archived on Zenodo with the MIT license. A development mirror exists at https://github.com/rub-sq/ReproBreak.

Reusable. The artifact is functional and verifiable with one command each. A smoke test checks the setup in seconds. `verify_claims.py` recomputes the paper's numbers from the database and compares them to the published values. Every one of the 449 breaks can be re-executed with `reproduce.py`: the test passes with the new locator and fails with the old one. A reduced validation scope of 8 curated breaks runs in 2 to 4 hours, the full scope covers all 449 breaks. Beyond that, the artifact is documented and structured for reuse. Dependencies are pinned, and the scripts can run in a provided container image. The README documents the full dataset schema, provenance, ethics, storage needs, and maps each supported and unsupported paper claim to commands with expected results. The paper's four application scenarios are mapped to concrete entry points, including an `overwrite` mode for validating locator repairs against ground truth. EXTENDING.md explains how to add repositories, frameworks, and reproduction environments.

Technology skills assumed by the reviewer evaluating the artifact and hardware requirements

Skills: basic command line, running Python scripts with uv or pip, and basic Docker. SQL helps for exploring the dataset but is not required.

Hardware: a commodity machine with at least 8 GB RAM and 30 GB free disk. No GPU. Tested on macOS with Apple Silicon. x86-64 Linux is expected to work, Windows is untested. Internet access is required, because the reproductions clone GitHub repositories and pull Docker base images. No commercial services or API keys are involved.

Provenance

The artifact is available on Zenodo. The record contains the code and documentation (`reprobreak-code.zip`), the dataset (`ReproBreak-db.zip`, 375 MB download, 1.0 GB unpacked), the author version of the paper, and SHA-256 checksums. A development mirror is maintained at https://github.com/rub-sq/ReproBreak. The dataset is derived from the public E2EGit dataset (MSR 2025) and public GitHub repositories. It stores no personal data.

Instructions

1. Download `reprobreak-code.zip` and `ReproBreak-db.zip` from the Zenodo record. Unzip the code, then unzip the database into its `data/` directory.
2. Install the dependencies with `uv sync` or `pip install -e .`. Alternatively, build the provided container image. Setup takes well under 30 minutes (README, Section 1).
3. Run `uv run smoke_test.py`. Expected: `SMOKE TEST PASSED` within seconds. The full expected output is printed in the README.
4. Run `uv run verify_claims.py`. Expected: `ALL CHECKS PASSED`. This confirms the paper's dataset numbers and the derivable columns of Tables 1 and 2.
5. Reproduce the paper's example break: `uv run reproduce.py --locator_id 1224 --mode reproduce_break` (expected: test fails), then `--mode fixed` (expected: test passes). The first run of a project builds its Docker image in 15 to 45 minutes. Later runs reuse the cache. `--list` shows all 449 reproducible IDs.
6. Reduced scope: `uv run run_reduced_experiment.py` validates 8 curated breaks in both modes, with progress output and a summary table, in 2 to 4 hours. Full scope: `--all`. Claims the artifact does not support directly are documented with reasons in the README, Section 2.2.

Required tools: git, Docker with a running daemon, Python 3.11 or newer with uv or pip, and unzip. The dataset schema and the usage scenarios are documented in the README, Sections 3 and 4.